01 La idea y los objetivos

Qué es el proyecto, para quién sirve y qué busca lograr.

EL PROYECTO EN UNA FRASE

Una web para buscar u ofrecer arriendos compartidos.

Centraliza publicaciones y perfiles para facilitar la búsqueda de vivienda y roomies.

El problema que aborda

Buscar un lugar o un compañero de vivienda requiere comparar ubicación, precio, cupos y condiciones. La propuesta reúne esos datos en una plataforma para organizar mejor la búsqueda.

Usuarios principales

Quienes buscan: personas mayores de edad que necesitan arriendo o vivienda compartida.
Quienes ofrecen: personas con una vivienda, habitación o cupo disponible.

OBJETIVO GENERAL

Facilitar la búsqueda y oferta de arriendos compartidos mediante una plataforma web que centraliza publicaciones y datos de los usuarios.

Objetivos específicos clave

1. **Gestionar cuentas y perfiles** para identificar a quienes usan la plataforma.
2. **Crear y administrar avisos** de búsqueda u oferta de alojamiento.
3. **Facilitar la exploración** con filtros de ubicación, cupos, vivienda y mascotas.
4. **Integrar interfaz, API y base de datos** para consultar y guardar información.

Objetivos formulados a partir del código actual. Beneficio esperado: una búsqueda más ordenada; el repositorio no aporta métricas de impacto.

02 Cómo se usa y qué incluye

El núcleo visible es la gestión de publicaciones y del perfil.

01 **Crear cuenta**

02 **Explorar avisos**

03 **Revisar detalles**

04 **Gestionar datos**

Funciones con interfaz y llamadas a la API

Acceso	Registro con nombre, correo, contraseña y fecha de nacimiento; inicio y cierre de sesión. El registro exige al menos 18 años.
Exploración	Tarjetas con precio en CLP, ubicación y cupos; detalle del aviso. Filtros por ubicación, cantidad de personas, casa/departamento y mascotas, aplicados en el navegador.
Publicaciones	Crear avisos de tipo “busco” u “ofrezco”. En “Mis publicaciones” se puede consultar, editar y eliminar. Datos principales: título, precio, ubicación, cupos, vivienda y descripción.
Perfil	Consultar y editar edad, ocupación, ciudad, descripciones y foto. Muestra el nombre y el número de publicaciones del usuario.

Otras piezas: su estado real

Conexiones y match. La API permite enviar, aceptar o rechazar solicitudes y registrar su verificación. No se encontró una pantalla conectada a este módulo.

Imágenes y ubicación. La API contempla imágenes para avisos y autocompletado con Geoapify. El formulario actual no envía la imagen ni consume el autocompletado.

Chat y favoritos. Aparecen en el menú; no se encontró implementación funcional en los archivos revisados.

¿QUÉ SIGNIFICA “MATCH” AQUÍ?

Una solicitud de conexión aceptada. El código no calcula compatibilidad entre personas.

03 Cómo está construido

Una aplicación FastAPI reúne las pantallas y los módulos del servidor.



Ejemplo: al guardar un aviso, JavaScript envía un POST; FastAPI valida los datos, los guarda y devuelve una respuesta. Las fotos se almacenan en una carpeta del servidor.

Tecnologías: para qué sirve cada una

HTML / CSS / JS	Estructura, diseño e interacción; fetch comunica las pantallas con la API.
FastAPI / Uvicorn	FastAPI define las rutas y valida solicitudes; Uvicorn ejecuta el servidor.
PostgreSQL	Guarda los datos. psycopg accede a cuentas, avisos y perfil; SQLAlchemy se usa en conexiones.
Pillow / HTTPX	Pillow valida imágenes de nuevos avisos. HTTPX consulta Geoapify para sugerir ubicaciones.

Datos principales y sus relaciones

Usuario → publicaciones → imágenes: el usuario es autor de los avisos; cada aviso puede tener imágenes asociadas.

Usuario solicitante → conexión ← usuario receptor: la solicitud puede estar pendiente, aceptada o rechazada.

Conexión aceptada → match: al aceptar se crea el registro de match; la conexión también tiene una marca de verificación.

Rutas para ubicarte en el código

/auth/registro · /auth/login Cuentas y acceso.
/api/publicaciones · /api/mis-publicaciones Gestión de avisos.
/api/perfil Datos personales. **/conexiones** Solicitudes y matches.

Validaciones al crear avisos: campos obligatorios, precio no negativo y cupos positivos. La API acepta JPG, PNG o WEBP de hasta 5 MB. Conexiones bloquea autosolicitudes y duplicados pendientes o aceptados.

04 Cómo presentarlo con claridad

Guion sugerido de cinco minutos y respuestas para la defensa.

0:00 - 0:45	**Problema y propuesta.** “¿Hay cupo? reúne avisos y perfiles para facilitar la búsqueda y oferta de vivienda compartida”.
0:45 - 1:30	**Objetivos.** Menciona el objetivo general y cuatro ideas: cuentas, publicaciones, filtros e integración de datos.
1:30 - 3:00	**Recorrido del usuario.** Muestra acceso, inicio, detalle de un aviso, creación o edición y perfil. Abre un detalle antes de usar los filtros.
3:00 - 4:00	**Arquitectura.** Explica el recorrido navegador → API → base de datos y la relación entre usuario, publicación y conexión.
4:00 - 5:00	**Resultados y próximos pasos.** Presenta el alcance del prototipo, las pruebas del módulo de conexiones y las integraciones pendientes.

Preguntas que debes poder responder

¿Qué es CRUD? Crear, consultar, actualizar y eliminar datos; aquí se aplica a las publicaciones.

¿Qué diferencia una conexión de un match? La conexión es la solicitud; un match se registra cuando se acepta.

¿Está terminado? Es un prototipo académico con módulos implementados y trabajo de integración pendiente.

EVIDENCIA DE PRUEBAS

3 pruebas automatizadas aprobadas con SQLite: flujo de solicitud/match/verificación, rechazo de autosolicitud y duplicados, y filtros. No validan la aplicación completa ni la base remota.

Prioridades para continuar

1. Reforzar contraseñas, sesiones y permisos; retirar credenciales del código.
2. Unificar los modelos y esquemas de datos entre publicaciones y conexiones.
3. Completar pantallas de conexiones, envío de imágenes y autocompletado.
4. Probar los flujos completos con la base real y actualizar la documentación.

Cierre sugerido: “El proyecto ya tiene una base para centralizar avisos de vivienda compartida. El siguiente paso es consolidar su integración, seguridad y validación completa”.

Base consultada: README.md; main/main.py; auth_service/main.py; match_conexion_service/{routes,models,database}.py; JavaScript de PaginaPrincipal, Creacion_De_publicacion y PerfilU; tests/test_match_conexiones.py. La guía manual contiene casos planificados, no resultados aprobados. Revisión de código al 04/11/2026; sin ejecutar la app ni consultar la base remota.